Department of Information Technology
College of Engineering, Guindy
Anna University, Chennai-25

LONG-TERM MEMORY ENGINE FOR AI AGENTS

Second Project Review

GUIDE:

DR. BAMA SRINIVASAN
DEPARTMENT OF INFORMATION TECHNOLOGY
CEG, ANNA UNIVERSITY

TEAM MEMBERS:

M. KRITHIKRAJ, 2023115004
ALSTON S, 2023115006
ARYAN I, 2023115021
HARI SRINIVASAN, 2023115022

PROBLEM STATEMENT

- An AI agent starts every session with a blank memory. Instructions, decisions and corrections from earlier sessions are gone unless the user repeats them.
- Pasting the whole history back into the prompt does not scale. Models make poor use of information in the middle of a long input [1], and each answer gets slower and more expensive.
- When something changes, the agent cannot tell the new value from the old one, so it can act on outdated information.
- An agent needs a memory that records what the user said and when it changed, and returns only the parts relevant to the current question, each with its source message. The LongMemEval benchmark [2] measures this.

This project builds that memory as a service for AI agents, connected here to Pi, an open-source coding agent that runs in the terminal.

LITERATURE SURVEY (1/2)

Work	How it works	Reported results	Limitation
MemGPT, Packer et al., 2023 [3]	Splits the prompt into a working area and outside storage. The model moves data between them by calling functions.	92.5% on a deep-memory recall test with GPT-4, against 32.1% for GPT-4 reading a summary.	Memory changes only when the model makes the right call. No speed or cost figures.
Zep / Graphiti, Rasmussen et al., 2025 [4]	A knowledge graph whose edges carry the time they became true and stopped being true. A contradiction closes the old edge.	71.2% on LongMemEval with gpt-4o, against 60.2% for the full 115k-token history.	A single run with one setup. The graph database is the only store.
Mem0, Chhikara et al., 2025 [5]	An LLM extracts candidate memories, compares each with similar stored ones and chooses add, update, delete or no change.	Score 66.9 from an LLM judge on the LoCoMo conversation benchmark, against 72.9 for the full history, but about 12 times faster.	One benchmark and an unnamed judge model. The open-source release has since dropped update and delete.

LITERATURE SURVEY (2/2)

Work	How it works	Reported results	Limitation
Supermemory, Daga et al., 2026 [6]	Small memories linked to the text they came from, with "updates", "extends" and "derives" links and two timestamps.	Scores on LongMemEval-S (the standard-size version), from the company's own run.	Closed source. The report is not peer reviewed and some of its metrics are not defined.
Hindsight, Latimer et al., 2025 [7]	Retain, recall and reflect operations. Recall merges four kinds of search by rank and trims the result to a size limit.	LongMemEval and LoCoMo scores from the authors' own runs.	Not peer reviewed. No repeated runs or component-by-component tests, and the size limit is not stated.
Full history in the prompt, Liu et al., 2024 [1]	No separate memory. The whole conversation goes into the model's input.	Accuracy drops when the needed passage sits in the middle of a long input.	Slower and costlier as history grows. Nothing marks a value as outdated.

CHALLENGES / ISSUES IDENTIFIED: GAPS IN THE LITERATURE

Gap in existing work	Seen in	How we address it
G1. The model decides how memory changes, so one wrong call silently loses or overwrites a memory.	MemGPT [3], Mem0 paper [5]	Changes follow fixed rules written in code, not a model decision.
G2. Changed values are overwritten or deleted, so nobody can see what used to be true.	Mem0 [5], MemGPT [3]; Zep [4] keeps history only in its graph	Every change is a new version that records when it was valid.
G3. A stored memory is hard to check against what the user actually said.	MemGPT [3], Mem0 [5]; Supermemory [6] is closed	Every memory cites an exact quote, matched against the original message.
G4. Extraction depends on large hosted models, which adds cost and sends user data to a third party.	Zep [4], Mem0 [5]	A small fine-tuned model runs on our own machine.
G5. Results are hard to compare: single runs, company-run benchmarks, different judges.	Zep [4], Mem0 [5], Supermemory [6], Hindsight [7]	We publish our test cases and are running LongMemEval-S with fixed settings.

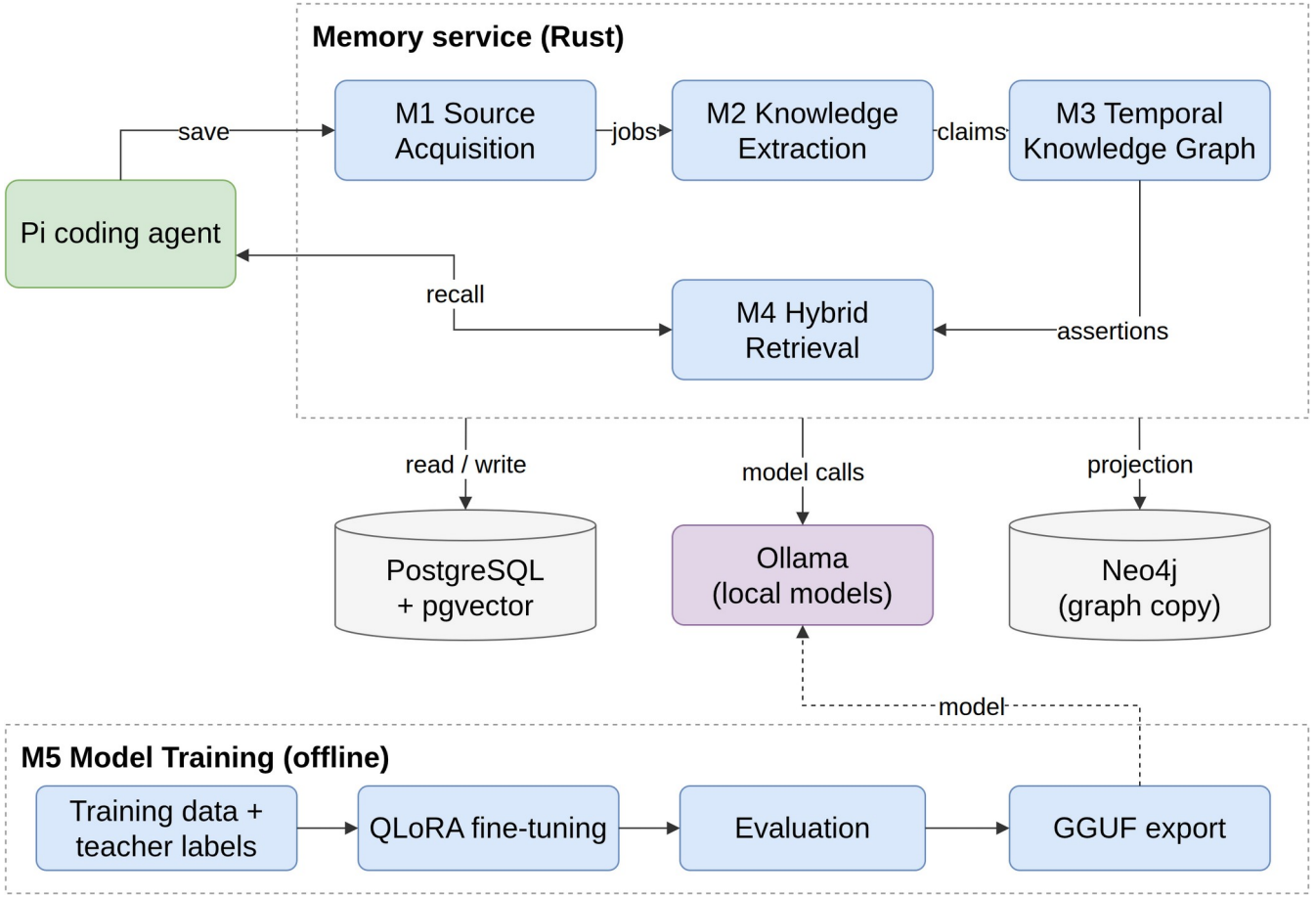
OBJECTIVES OF THE PROJECT

Objective	Answers
Keep useful information for an AI agent across sessions.	Problem
Apply changes with fixed rules in code, and keep earlier versions with the dates they were valid.	G1, G2
Link every stored memory to the exact message it came from.	G3
Fine-tune a small local model to extract memories, combine them and answer with citations.	G4
Return only the relevant memories, within a fixed size limit for the prompt.	Problem
Measure accuracy with repeatable tests and a public benchmark.	G5

KEY TERMS

Term	Meaning	Example
Source event	A message or tool result, saved exactly as received.	"I live in Chennai." (1 Oct)
Assertion	A statement the model extracts from source events, kept with the quote it came from.	Maya's home city is Chennai.
Slot	One attribute of one subject. A single-value slot has one current value.	Maya's home city
Correction	A new value that replaces the old one. The old one stays as history.	Bengaluru replaces Chennai.
Inference	A statement built from two or more assertions.	Maya works remotely as an engineer at NovaTech.
Stale	An inference whose supporting assertion has changed, so it is no longer used.	If Maya leaves NovaTech
Namespace	One user's or project's separate memory.	user:maya
Token budget	The most text the memory may add to the agent's prompt.	2,048 tokens

SYSTEM ARCHITECTURE

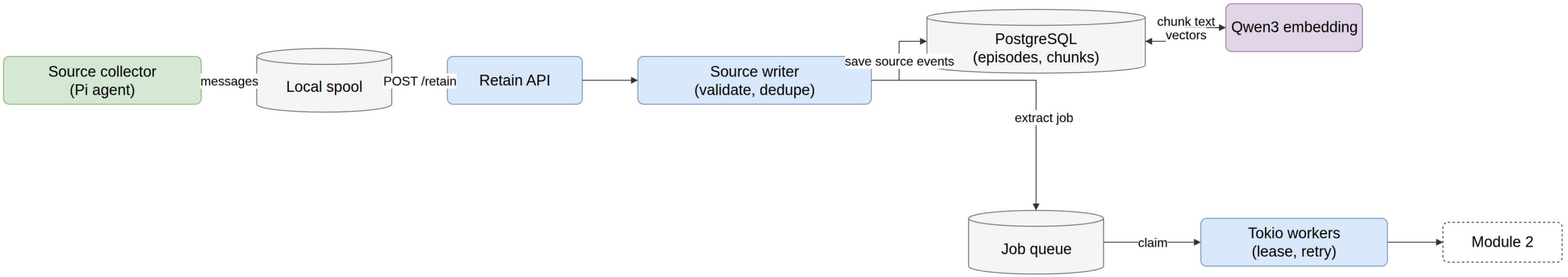


M1 saves messages, M2 extracts assertions, M3 keeps every version, M4 finds and packs memories for the agent, and M5 trains the extraction model offline.

EXAMPLE WALKTHROUGH

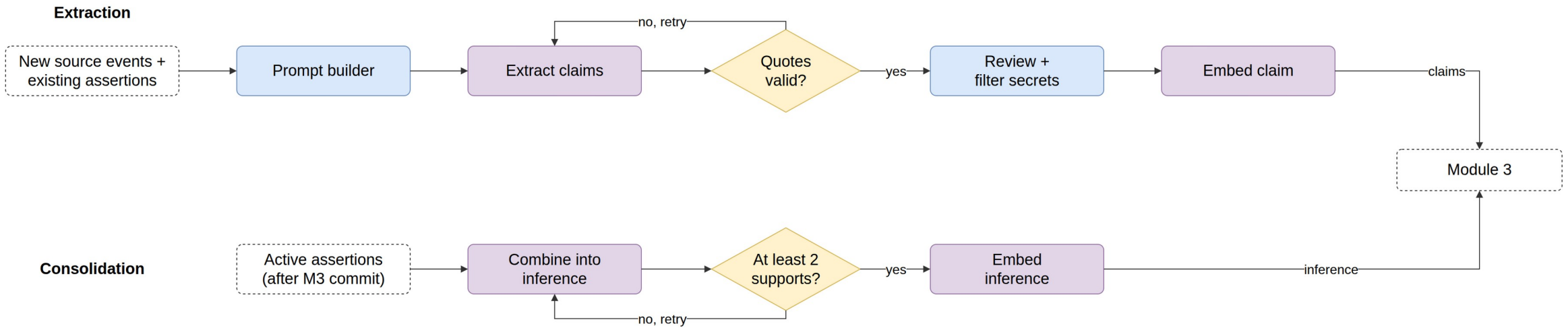
Input	1 Oct: "I live in Chennai." 10 Oct: "Bengaluru is now my home city." 13 Oct, new session: "Where do I live now?"
M1 Save	Both messages are saved exactly as sent, with speaker and time, and an extraction job is queued.
M2 Extract	The model from Module 5 returns two assertions, each with its quote: home city Chennai, then home city Bengaluru, marked as a correction.
M3 Update	Bengaluru supersedes Chennai. Chennai is kept, valid from 1 to 10 Oct, and Bengaluru is current from 10 Oct.
M4 Recall	For the 13 Oct question, search finds the Bengaluru assertion and passes it to Pi with its quote. Pi answers: "Bengaluru, since 10 October. Before that, Chennai."

MODULE 1: SOURCE ACQUISITION AND PROCESSING



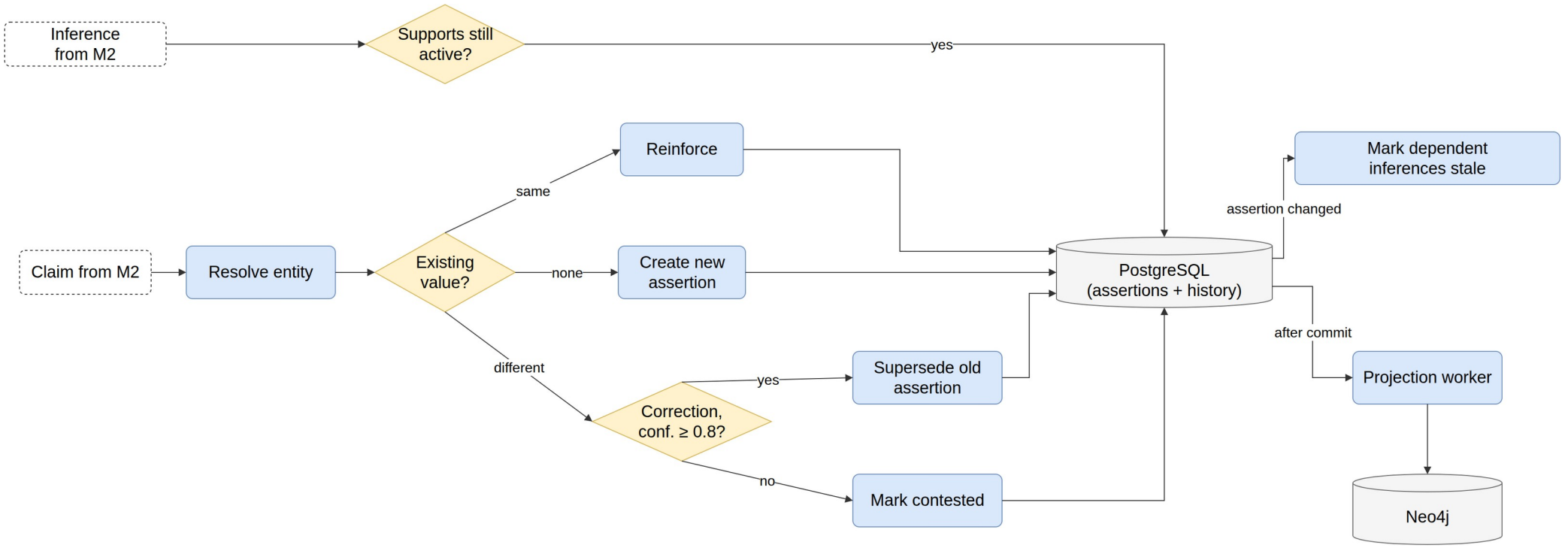
- Pi first writes each message to a local file queue, so nothing is lost if the memory service is down.
- The service's save endpoint stores them as source events, together with a job for Module 2. Sending the same request twice does not create duplicates.
- Messages are split into chunks and turned into vectors for search. Background workers pick up jobs and retry failures up to five times.

MODULE 2: KNOWLEDGE EXTRACTION AND CONSOLIDATION



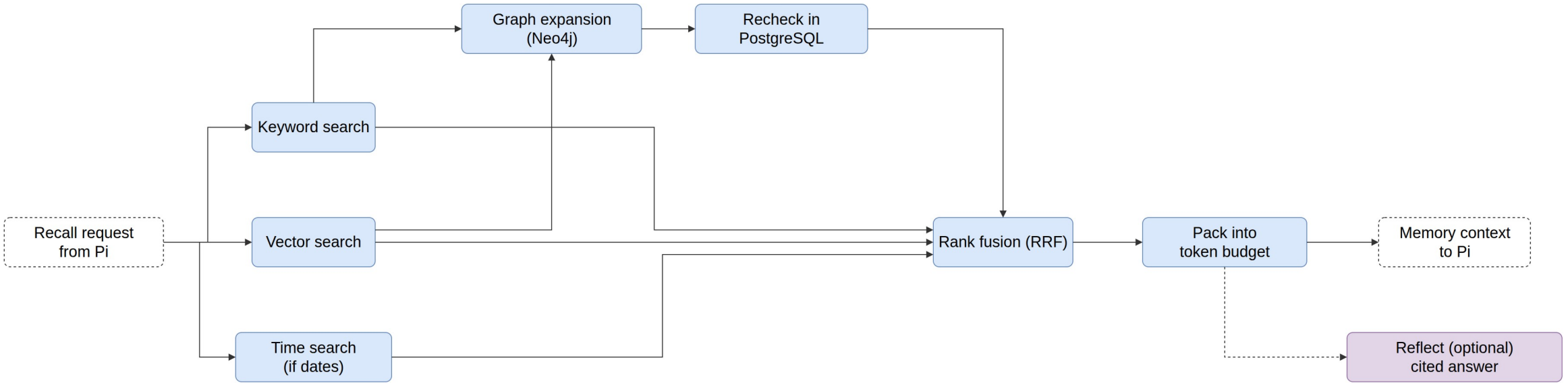
- Extraction: the fine-tuned model turns new source events into assertions, each with an exact quote. A quote that is not in the message is rejected, and the model tries again up to two times.
- A review step checks that a correction is really stated in the text and removes secrets.
- Consolidation: after Module 3 saves them, the model may combine two or more assertions about one subject into an inference.

MODULE 3: TEMPORAL KNOWLEDGE & GRAPH MANAGEMENT



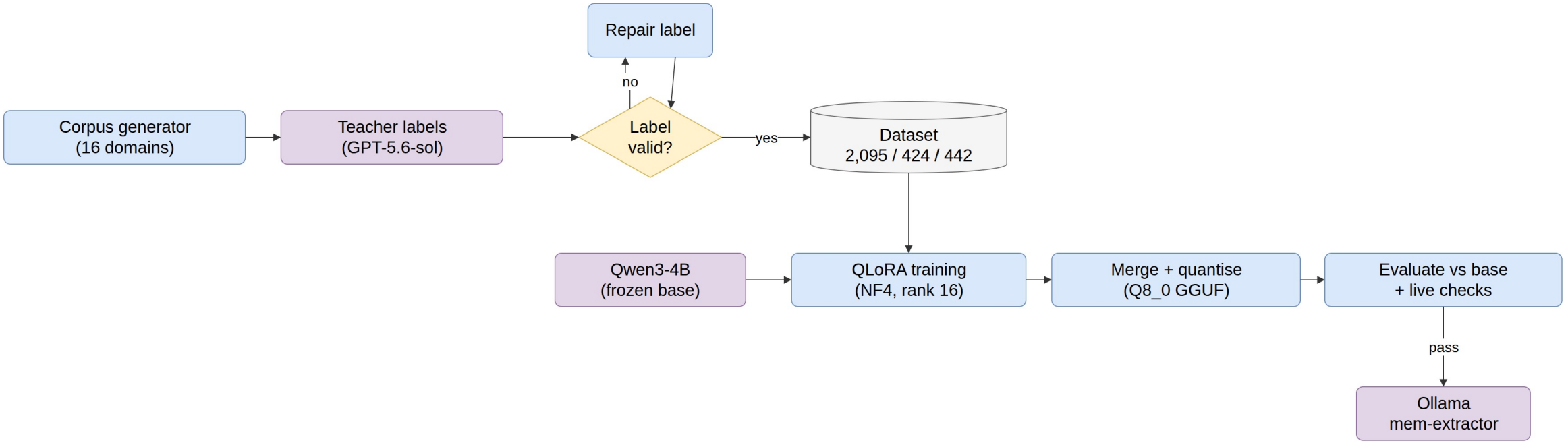
- The only module that writes assertions. In a single-value slot, a new value supersedes the old one only if it is a reviewed correction and the model's confidence is at least 0.8. Otherwise both are marked contested: kept, but neither is current.
- Replaced values keep the dates they were valid, and every change is copied to Neo4j for graph search.

MODULE 4: HYBRID RETRIEVAL AND GROUNDED REASONING



- Before Pi answers, it asks for memory. Keyword search, vector search and, for questions about dates, time-based search run together.
- Top hits are expanded through Neo4j and checked again in PostgreSQL. The lists are merged by rank and packed into the token budget, each with its quote.
- An optional step writes a full answer that cites the memories it used.

MODULE 5: MODEL TRAINING AND EVALUATION



- Runs offline. Short synthetic conversations from 16 domains are labelled by a larger model, GPT-5.6-sol. Live chats are never used for training.
- Qwen3-4B is fine-tuned on those labels and served to Modules 2 and 4 through Ollama, but only after it beats the base model on held-out tests.

ALGORITHMS OR METHODOLOGY

Algorithm	Purpose	Module
1. Priority-based context selection	Picks the existing assertions that matter most and fit the model's input.	M2
2. Entity resolution and versioning	Matches the subject, then handles repeats, corrections and conflicts.	M3
3. Consolidation and invalidation	Builds inferences and marks them stale when their support changes.	M2, M3
4. Hybrid retrieval and graph traversal	Finds memories by keyword, meaning, date and graph link.	M4
5. Rank fusion and context packing	Merges the search rankings and fits the best memories into the token budget.	M4
6. Source-grounded answering	Answers only from retrieved memories and checks every citation.	M4

ALGORITHMS OR METHODOLOGY

1. Priority-based context selection

Input: existing assertions, new messages, input limit

IF all existing assertions fit:

    RETURN all of them

Score each assertion by how often its subject and value appear in the new messages

Sort by score, newest first on ties

Find the longest sorted prefix that fits, by binary search

RETURN that prefix in its original order

2. Entity resolution and versioning

Input: a new assertion and the stored memory

Match the subject by type and name, or a unique alias; otherwise create it

IF the same assertion exists:

    attach the new quote

ELSE IF the slot has no conflicting value:

    store it as active

ELSE IF reviewed correction AND confidence ≥ 0.8:

    replace the old value, keep it as history

ELSE:

    mark both values contested

ALGORITHMS OR METHODOLOGY

3. Consolidation and invalidation

Input: active assertions about the changed subjects

Ask the model to combine related assertions into inferences

Keep an inference only if it has two or more supports

Check the support IDs and the subject

Check that every support is still active, then save with links

WHEN a support changes or is retracted:

    find inferences that depend on it

    mark them stale

4. Hybrid retrieval and graph traversal

Input: a question and an optional date range

Work out the dates a question refers to, if any

Run keyword search [8] and vector search [9]

IF there is a date range:

    add assertions valid in that range

Follow graph links from the matches for up to two steps

Drop results outside their validity or too far in meaning

RETURN the candidate lists

ALGORITHMS OR METHODOLOGY

5. Rank fusion and context packing

Input: ranked candidate lists and a token budget

FOR each memory:

    score = Σ 1 / (60 + rank) over all lists [10]

Sort by score, then by ID

FOR each memory in order:

    build a block with the assertion and its quotes

    add it if it fits the budget

    stop at the requested count

RETURN the packed context

6. Source-grounded answering

Input: a question and the retrieved memories

IF nothing was retrieved:

    RETURN "insufficient evidence"

Ask the model to answer only from the retrieved memories

Allow citations only to retrieved IDs

Check the output format and every cited ID

IF it claims an answer without citations:

    reject it

RETURN the answer with its citations

TOOLS AND TECHNOLOGIES (1/2)

Tool	Role
Rust, Tokio, Axum and SQLx	The memory service: API, background workers and database access.
PostgreSQL 17 with pgvector [8], [9]	Stores sources, assertions, history and jobs; keyword and vector search.
Neo4j 5.26 [11]	Graph copy of assertions and subjects, used to widen recall.
Ollama	Runs the memory model and the embedding model on our machine.
Qwen3-Embedding-0.6B [12]	Turns messages, assertions and questions into vectors.

TOOLS AND TECHNOLOGIES (2/2)

Tool	Role
Fine-tuned Qwen3-4B-Instruct-2507	Extracts assertions, combines them and answers with citations.
Pi coding agent [13]	The agent we connect to: it sends messages and receives recalled memory.
Python, PyTorch, Transformers, PEFT, bitsandbytes	Dataset preparation and QLoRA, a low-memory fine-tuning method [14].
llama.cpp and GGUF	Converts and compresses the trained model so Ollama can run it.
React, TypeScript, Vite and Bootstrap	Dashboard, session controls and the memory graph view.
Docker Compose, Caddy and ttyd	Runs the services and gives browser access to the app and the Pi console.

DATASET (1/3)

Fine-tuning dataset for the Module 5 model (Qwen3-4B-Instruct-2507)

Aspect	Details
Scope	Started with coding conversations, then grew to 16 domains such as family, food, travel, education, work and software.
Source	Synthetic conversations labelled by GPT-5.6-sol, checked by a second model and cleaned by us.
Size	2,961 task examples: 2,095 for training, 424 for validation and 442 for testing.
Tasks	Extract assertions, combine related assertions, and answer questions with citations.

Training example

Assistant: "You could learn violin this winter."
User: "No, that is not my plan. I am learning watercolor painting."
Expected memory: The user is learning watercolor painting.

DATASET (2/3)

External test dataset

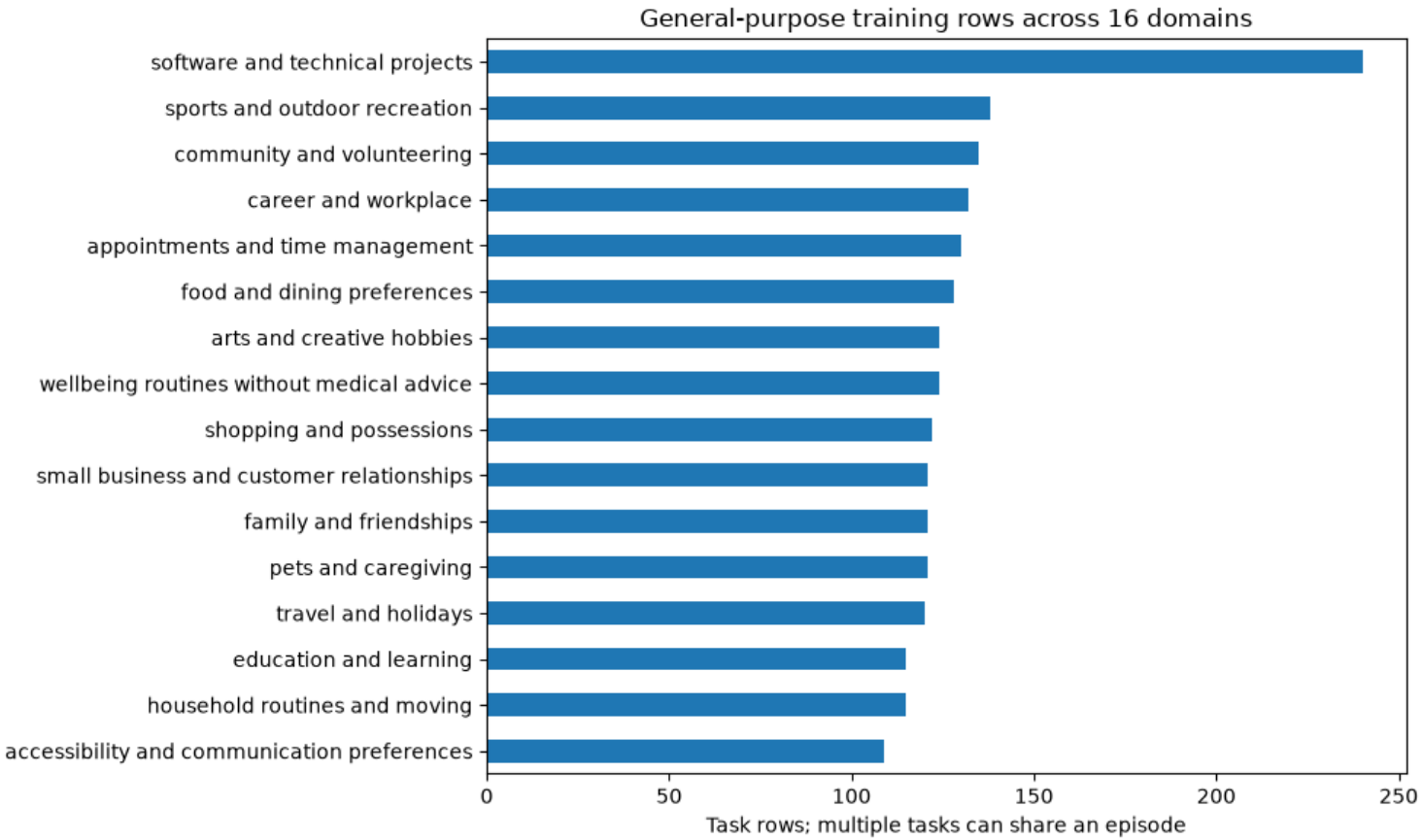
Aspect	Details
Source	LoCoMo [15]: long conversations spread over many sessions.
Size	16 conversation segments from 10 conversations, with 40 reference assertions.
Purpose	Check whether the model extracts useful assertions from conversations outside our training data.

Test example (LoCoMo)

Caroline: "Did you make that plate?"
Melanie: "Yes, during yesterday's pottery class. I find pottery relaxing and creative."
Caroline: "I haven't tried pottery. I've been painting and recently finished a picture."
Melanie: "What inspired your painting?"
Caroline: "A visit to the beach last week inspired it."

Expected memory: Melanie made a plate in pottery class. Caroline painted a picture inspired by her beach visit.

DATASET (3/3)



Of the 2,095 training rows, software and technical projects has 240; the other domains have about 110 to 140 each.

OUTPUT SCREENSHOTS (1/2)

Demo: two Pi sessions in different project folders, sharing one memory

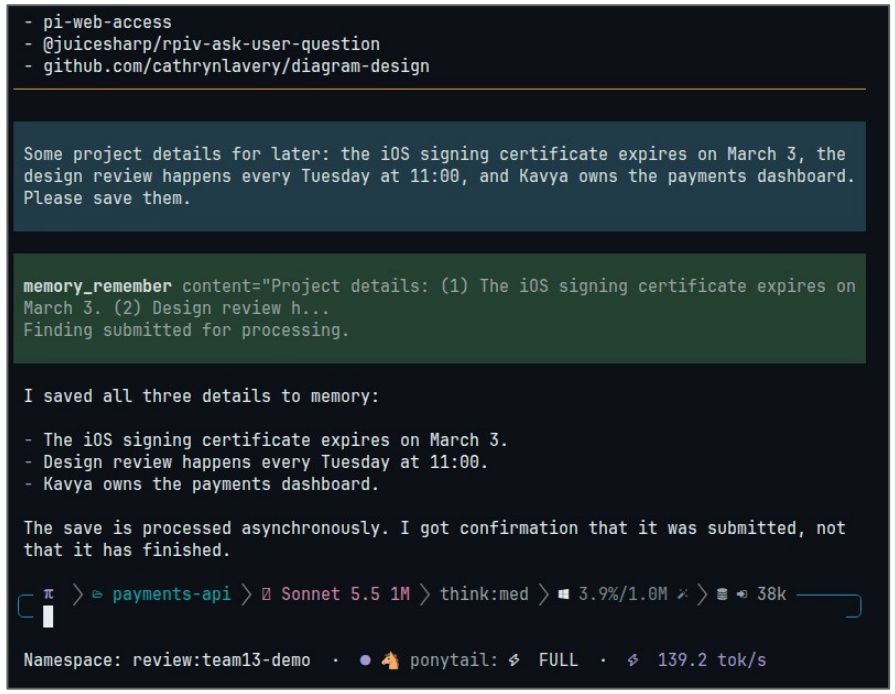


Figure 1. Recording: the user gives a certificate expiry date, a weekly review time and the dashboard owner, and Pi saves them to memory.

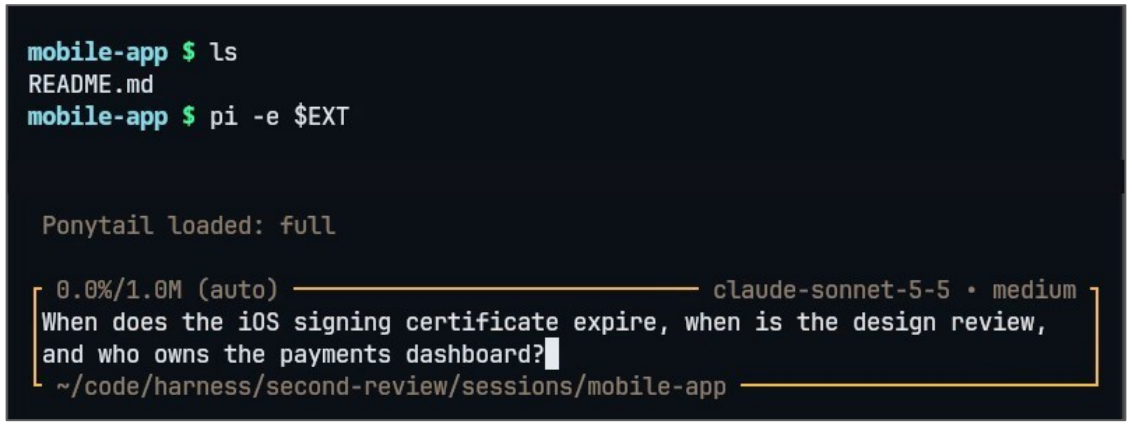


Figure 2. New session: a fresh session in another folder asks for the same three details without repeating them.

OUTPUT SCREENSHOTS (2/2)

The new session answers from memory, and the graph keeps the correction

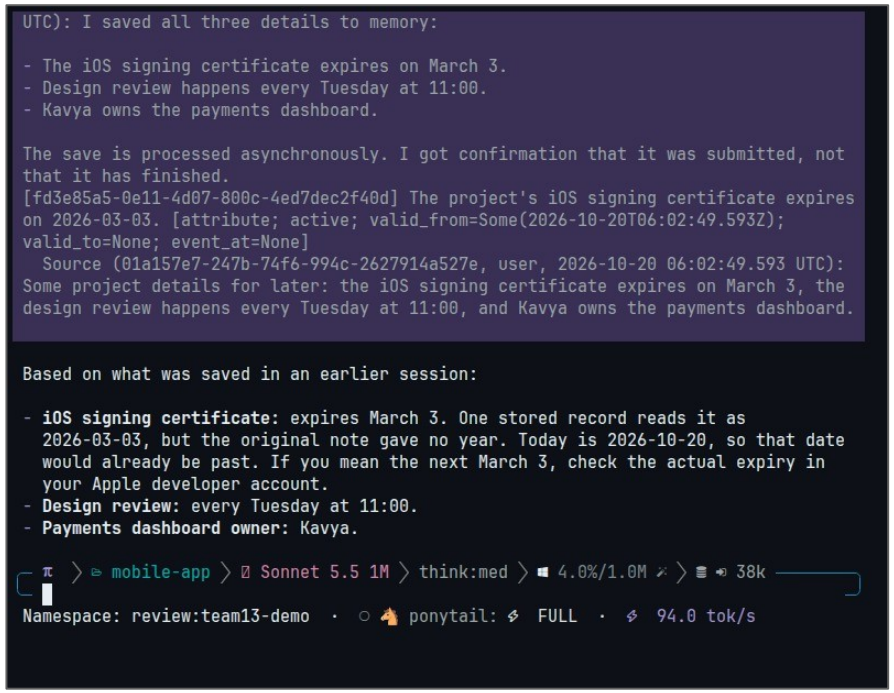


Figure 3. Recall: the memory from session 1 is added to the prompt, and Pi answers March 3, Tuesday at 11:00 and Kavya.

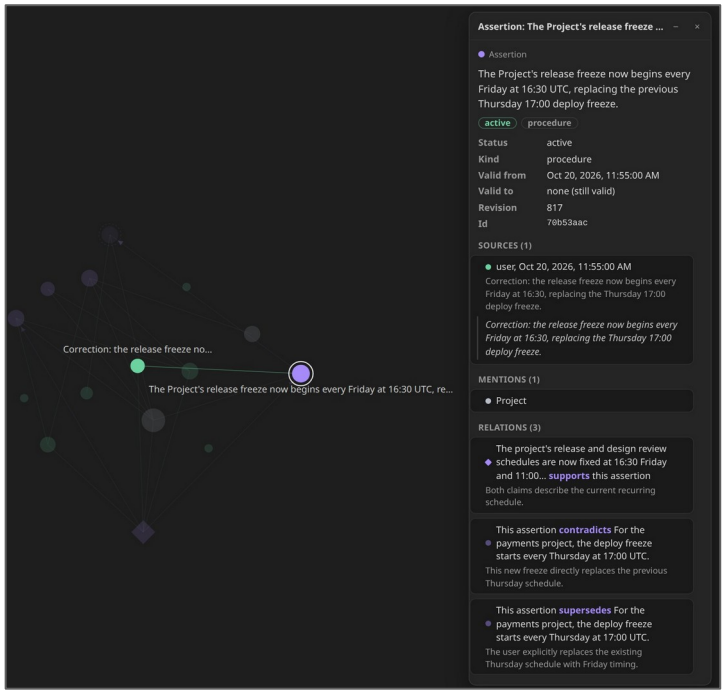


Figure 4. Correction: the user later moved the release freeze from Thursday 17:00 to Friday 16:30. The graph shows Friday as current, linked to the old value.

RESULTS AND OBSERVATIONS (1/3)

A corrected instruction carried into a new session.

In a three-session coding demo the required output changed from milliseconds to microseconds. In the next session the agent used microseconds without being told again.

Changing one assertion left the others alone.

A test conversation gave a person's city and food preference, then changed only the city. The system used the new city, kept the old one as history and still knew the food preference.

Every stored assertion can be traced to its message.

A database audit found that all 142 stored quotes match their original messages exactly.

Related assertions were combined.

The same audit found 8 inferences, each built from at least two assertions.

RESULTS AND OBSERVATIONS (2/3)

Base Qwen3-4B (before) and our fine-tuned model (after), on 64 held-out tasks and 16 LoCoMo samples

Measurement	Before	After
Correctly combined assertions (inferences)	1/16	13/16
Correct answers from supplied memories	10/16	15/16
Expected assertions recovered	50/55	41/55
Extracted assertions supported by the source	92.1%	90.5%
Corrections handled correctly	8/8	8/8
Outputs passing format checks	64/64	63/64
Expected assertions recovered (16 external samples)	16/40	13/40

Combining went from 1/16 to 13/16 and answering from 10/16 to 15/16, but extraction now misses more assertions.

RESULTS AND OBSERVATIONS (3/3)

The full benchmark has not been run yet.

Until LongMemEval-S is done, we cannot claim the memory improves answers over long histories.

Dates are still a weak point.

Some relative dates were read wrongly: "tomorrow" in a message sent on June 1 was stored as June 3.

The first model was too small.

Our first choice, Qwen3-1.7B, did well on synthetic examples, but only 18 of 33 real-style conversation segments passed validation. Most failures were wrong sources, quotes that did not match and invalid IDs. So we widened the data, tightened the output format and moved to Qwen3-4B.

STATUS AND PENDING WORK

Module	What works now	Pending	Status
M1 Source Acquisition	Saves Pi messages and tool results; handles duplicates, offline queueing and retries. Covered by Pi integration tests.	Access control for the API; tested backup and restore.	Implemented
M2 Knowledge Extraction	Extracts assertions with quotes, reviews corrections, builds vectors and inferences. Covered by tests.	Fewer missed assertions, better dates and sources; measure the review step.	Implemented; quality work pending
M3 Temporal Graph	Matches subjects, applies corrections, keeps history, marks stale inferences, updates Neo4j. Tests for each.	Unclear updates, aliases and long chains of corrections.	Implemented; wider testing pending
M4 Hybrid Retrieval	Keyword, vector, time and graph search; rank fusion, packing, recall in Pi and cited answers.	Full LongMemEval-S run, with and without graph search and inferences.	Implemented; benchmark pending
M5 Model Training	Qwen3-4B fine-tuned with QLoRA and served by Ollama. Tested on 64 internal and 16 external cases.	Fix the extraction drop and date errors; more external tests and human review.	Trained; evaluation ongoing

CONCLUSION AND NEXT STEPS

- All five modules are built. An agent can save what it is told, recall it in a new session and show the message each memory came from.
- Corrections replace old values without erasing them, and unrelated memories stay as they were.
- Fine-tuning raised correct inferences from 1/16 to 13/16 and correct answers from 10/16 to 15/16, but extraction dropped from 50/55 to 41/55.
- Next: fix extraction and date handling, run the full LongMemEval-S benchmark, and add external tests with human review.

REFERENCES (1/2)

[1] N. F. Liu et al., "Lost in the middle: How language models use long contexts," *Trans. Assoc. Comput. Linguist.*, vol. 12, pp. 157–173, 2024.

[2] D. Wu, H. Wang, W. Yu, Y. Zhang, K.-W. Chang, and D. Yu, "LongMemEval: Benchmarking chat assistants on long-term interactive memory," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2025, pp. 86809–86836.

[3] C. Packer et al., "MemGPT: Towards LLMs as operating systems," 2023, *arXiv:2310.08560*.

[4] P. Rasmussen, P. Paliychuk, T. Beauvais, J. Ryan, and D. Chalef, "Zep: A temporal knowledge graph architecture for agent memory," 2025, *arXiv:2501.13956*.

[5] P. Chhikara, D. Khant, S. Aryan, T. Singh, and D. Yadav, "Mem0: Building production-ready AI agents with scalable long-term memory," in *Proc. 28th Eur. Conf. Artif. Intell. (ECAI)*, 2025, pp. 2993–3000, doi: 10.3233/FAIA251160.

[6] S. Daga, S. Sreedhar, and D. Shah, "LongMemEval," Supermemory Research, May 2026. Accessed: Oct. 6, 2026. [Online]. Available: https://supermemory.ai/research/longmembench/

[7] C. Latimer et al., "Hindsight is 20/20: Building agent memory that retains, recalls, and reflects," 2025, *arXiv:2512.12818*.

[8] PostgreSQL Global Development Group, "Full text search," in *PostgreSQL 17 Documentation*, ch. 12, 2026. Accessed: Oct. 6, 2026. [Online]. Available: https://www.postgresql.org/docs/17/textsearch.html

REFERENCES (2/2)

[9] A. Kane et al., "pgvector: Open-source vector similarity search for Postgres," GitHub repository, 2026. Accessed: Oct. 6, 2026. [Online]. Available: https://github.com/pgvector/pgvector

[10] G. V. Cormack, C. L. A. Clarke, and S. Büttcher, "Reciprocal rank fusion outperforms Condorcet and individual rank learning methods," in *Proc. 32nd Int. ACM SIGIR Conf. Res. Develop. Inf. Retrieval*, Boston, MA, USA, 2009, pp. 758–759, doi: 10.1145/1571941.1572114.

[11] Neo4j, Inc., "Query the database," in *Neo4j HTTP API Manual*, v5, 2026. Accessed: Oct. 6, 2026. [Online]. Available: https://neo4j.com/docs/http-api/5/query/

[12] Y. Zhang et al., "Qwen3 Embedding: Advancing text embedding and reranking through foundation models," 2025, *arXiv:2506.05176*.

[13] M. Zechner et al., "pi: AI agent toolkit," GitHub repository, 2026. Accessed: Oct. 6, 2026. [Online]. Available: https://github.com/earendil-works/pi

[14] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "QLoRA: Efficient finetuning of quantized LLMs," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 36, 2023, pp. 10088–10115.

[15] A. Maharana, D.-H. Lee, S. Tulyakov, M. Bansal, F. Barbieri, and Y. Fang, "Evaluating very long-term conversational memory of LLM agents," in *Proc. 62nd Annu. Meeting Assoc. Comput. Linguist.*, Bangkok, Thailand, 2024, pp. 13851–13870, doi: 10.18653/v1/2024.acl-long.747.